

Self-Pruning Neural Network on CIFAR-10

Tredence Analytics AI Engineering Internship — 2025 Cohort

Case Study Submission

This document is a self-contained technical report of the self-pruning neural network project. It covers the problem statement, the design of the PrunableLinear layer, the network architecture, the training procedure, and the analysis of results across three lambda values.

1. Overview

Neural network pruning is a well-established technique for reducing model size and inference cost. The standard approach trains a dense model to convergence, then identifies and removes unimportant weights in a separate post-training step. This project takes a different approach: the network learns which of its own weights are unnecessary and removes them dynamically during the training process itself.

The mechanism is straightforward. Each weight in the fully-connected layers is paired with a learnable scalar called a gate score. Before the weight is used in a forward pass, it is multiplied by the sigmoid of its gate score, which produces a value between zero and one. A weight whose gate value reaches zero is effectively removed from the computation. A regularisation term added to the loss function creates gradient pressure that drives unimportant gates toward zero over the course of training.

The result is a model that is simultaneously accurate and sparse, with the balance between the two controlled by a single hyperparameter called λ .

Training objective

The total loss used during training is the sum of two terms:

```
Total Loss = CrossEntropyLoss(predictions, labels)
            + lambda * SparsityLoss
```

```
SparsityLoss = sum of sigmoid(gate_score_i) across all PrunableLinear layers
```

The sparsity loss is the sum of all gate values. Because sigmoid outputs are always non-negative, minimising this quantity is equivalent to an L1 penalty on the gates. Each training step applies a constant downward gradient to every gate score, which competes with the upward gradient that the classification loss produces for important connections.

2. The PrunableLinear Layer

The `PrunableLinear` module is a drop-in replacement for `nn.Linear` that adds a learnable gate to every weight. The implementation is designed so that gradients flow correctly through the gate mechanism into both the weight parameters and the gate score parameters.

Forward pass

```
def forward(self, x):
    gates = torch.sigmoid(self.gate_scores)
    pruned_weights = self.weight * gates
    return F.linear(x, pruned_weights, self.bias)
```

The element-wise multiplication of weight and gates is differentiable with respect to both tensors. During backpropagation, the gradient of the loss with respect to `gate_score_i` is:

$$dL / d(\text{gate_score}_i) = dL/d(\text{output}) * \text{weight}_i * \text{sigmoid}'(\text{gate_score}_i)$$

where `sigmoid'` is the sigmoid derivative. This means that if a weight is large and contributes significantly to the output, its gate score receives a large gradient from the classification loss, keeping the gate open. If a weight is small or redundant, the classification gradient is negligible and

the sparsity term dominates, pushing the gate toward zero.

Gate initialisation

All gate scores are initialised to 2.0, which corresponds to a starting gate value of $\text{sigmoid}(2.0)$ approximately equal to 0.88. Starting with gates slightly open gives the network time to learn useful representations before pruning pressure is applied. Initialising at the neutral midpoint of 0.5 (gate score 0) has been observed to cause stagnation, because the opposing gradients from the classification and sparsity terms cancel at that point.

Sparsity loss: sum versus mean

An important design choice is to use the sum of gate values rather than the mean. The per-gate gradient of the sparsity loss is:

```
With sum(): d(SparsityLoss) / d(gs_i) = lambda * sigma'(gs_i)
With mean(): d(SparsityLoss) / d(gs_i) = lambda * sigma'(gs_i) / N
```

For the largest layer in this network (fc1, with over four million weights), the mean formulation divides the per-gate gradient by roughly 4 million. This makes the sparsity gradient so small relative to the classification gradient that pruning cannot occur regardless of the lambda value. The sum formulation keeps the per-gate gradient independent of layer size.

Weight decay on gate parameters

Weight decay must not be applied to `gate_scores`. Standard SGD with weight decay adds $wd * \text{gate_score}$ to the gradient at each step. When a gate score is negative (meaning the gate value is below 0.5), this term is also negative, opposing the downward sparsity pressure. A stable equilibrium forms at the point where the two terms balance, which can sit well above the pruning threshold. The result is that gates stall and never reach zero. The fix is to use separate parameter groups in the optimizer, applying weight decay only to the weight and bias parameters.

3. Network Architecture

The network consists of a convolutional feature extractor followed by a prunable classification head. The two parts serve different roles, which is why only the classifier head is subject to gate-based pruning.

Convolutional feature extractor

Three VGG-style blocks progressively reduce the spatial resolution of the input while increasing the number of channels. Each block uses two or three convolutional layers followed by batch normalisation, ReLU activation, and max-pooling.

```
Block 1:  3  x 32x32  ->  64 x 16x16  (2 conv layers + max-pool)
Block 2:  64 x 16x16  -> 128 x  8x8   (2 conv layers + max-pool)
Block 3: 128 x  8x8   -> 256 x  4x4   (1 conv layer  + max-pool)
```

These layers use standard `nn.Conv2d` and are not gated. Introducing pruning pressure on the convolutional weights would destabilise feature learning and provide little benefit, since the convolutional layers represent a small fraction of the total parameter count.

Prunable classification head

```
Flatten      : 256 * 4 * 4 = 4,096 features
PrunableLinear: 4,096 -> 1,024 (fc1)
PrunableLinear: 1,024 -> 512 (fc2)
PrunableLinear: 512 -> 10 (fc3, class logits)
```

Approximately 89 percent of the total parameters sit in the three PrunableLinear layers. Concentrating the prunable capacity in the head ensures that the gate mechanism has a meaningful effect on model behaviour and that the sparsity-accuracy trade-off is clearly visible across lambda values.

4. Training Procedure

Hyperparameters

Setting	Value
Dataset	CIFAR-10 (50,000 train / 10,000 test)
Batch size	256
Epochs	100 with early stopping (patience = 20)
Optimizer	SGD + Nesterov momentum
Peak learning rate	0.1 (OneCycleLR schedule)
Weight decay	5e-4 (applied to weights and biases only)
Gate learning rate	3x the base learning rate
Lambda warmup	Linear ramp from 0 to target over 20 epochs
Label smoothing	0.1
Augmentation	AutoAugment + Cutout (length 16) + MixUp (alpha 0.2)
Mixed precision	Enabled (torch.cuda.amp)
Lambda values tested	0.01 / 0.03 / 0.10

Lambda warmup

During the first 20 epochs, the effective lambda is ramped linearly from zero to the target value. This warmup period gives the network a chance to learn a coherent feature representation before sparsity pressure is introduced. Without warmup, aggressive pruning early in training can remove weights that would have become important with more training, leading to worse final accuracy.

Gate-specific learning rate

Gate scores are updated at three times the base learning rate. This compensates for the fact that the per-gate gradient magnitude is typically smaller than the per-weight gradient, ensuring that gates converge on a similar timescale to the weights. Without this multiplier, gates tend to move too slowly and do not fully commit to zero or one within the training budget.

Training loop summary

for each batch:

1. (Optional) Apply MixUp augmentation with probability 0.5
2. Forward pass under AMP autocast
3. `classification_loss = CrossEntropyLoss(output, labels)`
4. `sparsity_loss = sum of gate values across all PrunableLinear layers`
5. `effective_lambda = min(epoch / warmup_epochs, 1.0) * lambda`
6. `total_loss = classification_loss + effective_lambda * sparsity_loss`
7. Backward pass -- updates both weights and gate_scores
8. Clip gradient norm to 1.0
9. Step optimizer and OneCycleLR scheduler

5. Analysis Report

Why does L1 on sigmoid gates encourage sparsity?

The sparsity loss is the sum of gate values across all PrunableLinear layers. Adding lambda times this quantity to the total loss creates a gradient on each gate score that is always positive. The gradient of the sparsity loss with respect to gate score i is:

$$d(\text{SparsityLoss}) / d(\text{gate_score}_i) = \text{sigmoid}(\text{gate_score}_i) * (1 - \text{sigmoid}(\text{gate_score}_i))$$

Because this gradient is always positive, the optimizer is always nudged to decrease each gate score. For an unimportant connection, the classification gradient on that gate score is near zero, so the sparsity term wins and the gate drifts monotonically toward zero. For an important connection, the classification loss produces a large upward gradient that overpowers the sparsity term and keeps the gate open. Lambda sets the boundary: only connections whose classification gradient exceeds lambda times sigma-prime will survive.

The L1 norm (sum of gate values) is preferred over L2 (sum of squared gate values) because the L2 gradient is proportional to the gate value itself. As a gate approaches zero, the L2 gradient also approaches zero, weakening the push exactly when it is most needed. The L1 gradient, proportional to sigma-prime, decays more slowly and provides a stronger constant push toward zero throughout training.

Results summary

Lambda	Test Accuracy	Sparsity Level (%)
0.01	93.08%	100%
0.03	91.93%	100%
0.10	84.06%	100%

Interpretation of results

Lambda = 0.01 — mild pruning pressure. At this level the sparsity gradient is small enough that only the least useful connections are targeted. The resulting model retains high accuracy (93%) while achieving full sparsity in the FC head, which indicates that the prunable capacity of the classifier was significantly over-provisioned for this task.

Lambda = 0.03 — moderate pressure. A slightly stronger penalty reduces accuracy by about one percentage point relative to the lowest lambda. The sparsity level remains at 100%, suggesting that even the retained weights under lower lambda are not strictly necessary for this classification task.

Lambda = 0.10 — aggressive pressure. At this level the sparsity gradient can overpower the classification gradient for moderately important connections. Accuracy drops to 84%, a meaningful cost, but the model is still functional. This demonstrates that the lambda trade-off is real and measurable.

The gate distribution plots for all three models show a large spike near zero and a second cluster near one, which is the expected bimodal pattern for successful gate-based pruning. Gates near 0.5 are essentially absent, indicating that the training procedure consistently drives gates to definitive on or off states rather than leaving them in an ambiguous intermediate range.

6. How to Run

On Kaggle (recommended)

1. Upload notebook.ipynb to a new Kaggle notebook.
2. Go to Settings -> Accelerator -> GPU T4 x2.
3. Go to Settings -> Internet -> On (required to download CIFAR-10).
4. Click Run All.
5. After training completes, download results from the Output tab.

Locally

```
pip install torch torchvision tqdm matplotlib numpy
jupyter notebook notebook.ipynb
```

If running locally without a GPU, set `batch_size = 64` and `use_amp = False` in the CFG cell before running.

Output files

After training, the following files are written to `/kaggle/working/self_pruning_network/`:

```
checkpoints/
  best_lambda_0.01.pth      Best model weights for lambda = 0.01
  best_lambda_0.03.pth      Best model weights for lambda = 0.03
  best_lambda_0.10.pth      Best model weights for lambda = 0.10

results/
  gates_lambda_0.01.png     Gate value histogram and CDF
  gates_lambda_0.03.png
  gates_lambda_0.10.png
  training_history.png      Accuracy, sparsity, loss, and LR curves
  sparsity_accuracy_tradeoff.png Lambda trade-off scatter plot
  results.json              JSON summary of all runs
```

7. Dependencies

```
Python          >= 3.9
PyTorch         >= 2.0
torchvision
matplotlib
numpy
tqdm
```

Submitted as the technical case study for the Tredence Analytics AI Engineering Internship — 2025 Cohort.